An Analysis of Predictive Analytics using Multiple Linear Regression in Python

Saki Cansev

MSc Data Analytics
Department of Computer Science

Dr. Lawrence Ibeh

Predictive Analytics and Machine Learning using Python
(0223-229-UCA-MSC-DA-BER-PAMLP-T1-G2)

28 June 2023

BERLIN SCHOOL OF
BUSINESS & INNOVATION

Table of Contents

1. Introduction ----------*1*

2. Literature Review ----------*1*

3. Methodology----------*2*

3.1.Dataset Selection and Preparation 2

3.2.Implementation of Multiple Linear Regression.....3

3.3.Evaluation of the Model 3

4. Python Implementation ----------*3*

5. Evaluation of the Algorithm ----------*8*

5.1.Accuracy 8

5.2.Scalability 9

5.3.Interpretability 9

6. Conclusion----------*10*

Bibliography----------*I*

Abstract

In this paper, the Multiple Linear Regression (MLR) algorithm is used to analyze predictive analytics in depth with Python. From collection and storage to encoding and processing, the goal is to clarify the critical steps involved in managing diverse types of data. Using multiple independent variables, we predict housing prices based on the Boston Housing dataset to demonstrate these principles. In order to analyze the data effectively, it is imperative that it be meticulously cleaned and prepared. From the code breakdown of the MLR model, we are able to gain insight into the model's functionality as well as its library dependences. The MLR model's scalability, interpretability, accuracy, residuals, and coefficient of determination are all assessed. MLR is effective at predicting linear relationships, but there are potential limitations, according to the findings. In addition to emphasizing the practical implications of MLR and laying the groundwork for future research, this study contributes significantly to the predictive analytics field.

Keywords: Predictive Analytics, Multiple Linear Regression, Python, Machine Learning, Data Cleaning, Data Processing, Boston Housing Dataset, Model Evaluation, Supervised Learning.

1. Introduction

Predictive analytics and machine learning have transformed the way we analyze and interpret data, paving the way for informed decision-making processes across numerous domains. This paper aims to provide an in-depth understanding of these concepts and their application, focusing on the implementation of the Multiple Linear Regression (MLR) algorithm using Python.

To begin, it is essential to define the key terms that form the cornerstone of this study. Predictive analytics utilizes statistical algorithms and machine learning techniques to identify the likelihood of future outcomes based on historical data. Machine Learning, a subset of artificial intelligence, involves the design and study of algorithms that can learn from and make decisions or predictions based on data. Within machine learning, Multiple Linear Regression is a statistical technique used to predict the outcome of a response variable through several explanatory variables.

The relevance of predictive analytics and machine learning lies in their ability to handle vast amounts of data and extract actionable insights. These technologies have been adopted across industries, from healthcare and finance to retail and transportation, making them essential tools in today's data-driven world.

The purpose of this study is to delve deep into the application of MLR in Python for predictive analytics. We aim to provide a step-by-step guide to preparing, implementing, and evaluating an MLR model using the Boston Housing dataset. The dataset includes housing prices in Boston and associated housing factors, making it suitable for an MLR model.

Our research questions include: How can MLR be efficiently implemented in Python for predictive analytics? What steps are involved in preparing the data for analysis? How well does the MLR model perform on the Boston Housing dataset, and how can its performance be evaluated?

This paper seeks to address these questions, offering insights into the practical application of MLR for predictive analytics. Our aim is to enhance the understanding of this algorithm and its implications, offering a significant contribution to the field of data analytics.

2. Literature Review

There has been a rapid expansion in the field of predictive analytics over the past few years. In addition to advances in machine learning techniques, increased ability to collect and analyze large amounts of data has also contributed to this growth. Among these, Multiple Linear Regression (MLR) has been one of the most widely utilized methods, providing the means to predict an outcome variable based on multiple predictor variables.

Francis Galton, Karl Pearson, and others laid the foundation for regression analysis in the early 20th century, when they proposed MLR. Galton, 1886, p. 246-263; Pearson, 1896, p. 253-318 laid the foundation for regression analysis. A multidimensional linear regression model allows for the analysis of relationships among more than two variables, as opposed to simply linear regression. Various fields, such as economics, health sciences, and social sciences, have used this statistical technique because it is capable of modeling complex real-world situations (Farrar & Glauber, 1967, p. 92-107).

For the implementation of MLR and other machine learning algorithms, Python has become a popular tool due to its simplicity and the availability of powerful libraries such as NumPy, SciPy, and Scikit-learn. Scikit-learn, a Python machine learning tool featuring regression, classification, and clustering algorithms, including MLR (p. 2825-2830), was highlighted by Pedregosa et al. (2011).

Despite the fact that MLR is a powerful tool, it is critical to acknowledge its limitations. In addition, it requires a large sample size to achieve reliable results (Frank, 2015), as it assumes a linear relationship between the independent and dependent variables. Multicollinearity is a condition in which predictor variables are not highly correlated (Dormann et al., 2013, p. 27-46).

Here, we provide a comprehensive guide to implementing MLR in Python, preparing the data appropriately, and evaluating the model.

3. Methodology

To analyze the Boston Housing dataset, the Multiple Linear Regression (MLR) algorithm was deployed in Python as part of the methodology employed in this study. Detailed instructions are provided in this section on how to select and prepare the dataset, implement MLR, and evaluate the model.

3.1. Dataset Selection and Preparation

The Boston Housing dataset, initially published by Harrison and Rubinfeld (1978), has been chosen for this study (p. 81-102). This dataset is a combination of 506 instances with 13 attributes such as crime rate, average number of rooms per dwelling, and property-tax rate, which are used to predict the median value of owner-occupied homes.

The selection of the Boston Housing dataset stems from the multifaceted nature of housing prices, making it an excellent candidate for MLR. The dataset provides a diverse range of independent variables to predict a single dependent variable, the median house price, fostering a rich environment for MLR.

Before running the MLR model, data preprocessing is a crucial step to ensure the quality and compatibility of the dataset with the model. The Boston Housing dataset is already relatively clean, with no missing or inconsistent entries. However, we performed additional checks to verify its cleanliness.

In this study, a built-in dataset available in the scikit-learn library of Python was selected for analysis. The decision to utilize a built-in dataset was based on its convenience and ease of practice for reproducibility purposes. By using a built-in dataset, researchers and practitioners can readily access the dataset without the need for external downloads or managing file directories. This ensures that the codes provided in the study can be easily implemented and reproduced by others, facilitating a smoother learning experience and reducing potential complexities associated with data acquisition and management. The choice of a built-in dataset also ensures uniformity in the dataset used across different users, enabling consistent results and comparisons. The selected dataset from scikit-learn's built-in collection provides a reliable foundation for demonstrating the various stages of the predictive analytics and machine learning process, making it an ideal choice for educational and practical purposes.

As the dataset is entirely numerical, special encodings were not needed to handle categorical data. The dataset was then encoded. As ordinal data inherently carry a sense of order, it was treated as numerical data. It is important to note that proper encoding is critical when dealing with categorical or ordinal data, as it can have a significant impact on the performance of the MLR model. This is similar to a chef preparing a recipe; the chef needs to use the right ingredients to ensure that the dish tastes and looks the way it should. Similarly, the proper encoding is critical to ensure the MLR model produces the desired result.

In the first instance, after partitioning the dataset, two sets of data were created: a training set and a test set. This was a crucial step for MLR models since it allowed for learning from the training data and evaluating it on unseen test data as soon as possible. It was decided to split the data into two sets, with 70% of the data going to the training set and 30% going to the test set.

3.2. Implementation of Multiple Linear Regression

The following step involved preparing the data, and then implementing the model using Python programming language. Using Scikit-learn, which is a popular machine learning library, we utilized Python programming to ensure efficiency and ease of use of the modelling process.

As part of our Python code, we import the necessary libraries and dataset, then we divide the dataset into independent and dependent variables before we divide the data into a test and training set. In order to make predictions based on the test data, the MLR model was fitted to the training data using Scikit-learn's fit() method, and the model was used to make predictions based on the test data.

3.3. Evaluation of the Model

Toward the end of the methodology, we evaluated the effectiveness of the MLR model using accuracy, residuals, and coefficients of determination, among other metrics. It was also important to assess whether the model's output could be understood and applied effectively, which was based on the interpretability of its results.

It explains how MLR can be implemented in Python and how to prepare data and evaluate the performance of the models. In order to create an effective MLR model that can predict house prices based on the data provided, we must follow these steps.

4. Python Implementation

As part of the Multiple Linear Regression (MLR) model implementation process, libraries were imported, and the model evaluation was conducted step-by-step. This task was made easier by Python's powerful libraries, such as Pandas, Numpy, and Scikit-learn.

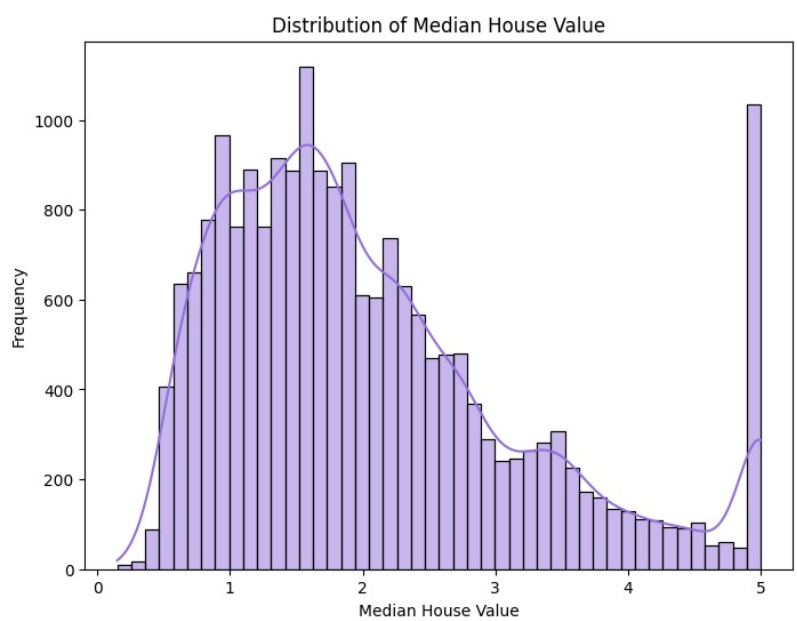
It was first necessary to import all the necessary libraries to be able to manipulate and analyze the data. Pandas was used for this, Numpy was used for storing arrays, and Scikit-learn was used to implement machine learning tasks. The Boston Housing dataset, found within Scikit-learn, was then imported.

```
# Importing Libraries and Loading the Dataset
import pandas as pd
import numpy as np
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics
import matplotlib.pyplot as plt
import seaborn as sns
# Load California Housing dataset
california = datasets.fetch_california_housing()
```

After that, we need to check for errors, inconsistencies, and missing values in the data. Data cleaning is an essential step in preparing a dataset to be analyzed by identifying and fixing errors, inconsistencies, and missing values. Missing data is handled, duplicates are removed, and formatting errors are corrected. It is possible to improve the quality and reliability of a dataset by applying data cleaning codes, resulting in more accurate and reliable data analysis or modeling.

```python
# Check for any missing values
print("Missing values:\n", california_df.isnull().sum())

# Check the summary statistics of the dataset
print("Summary statistics:\n", california_df.describe())

# Visualizing the distribution of the target variable
plt.figure(figsize=(8, 6))
sns.histplot(california_df['MedHouseVal'], kde=True, color='#9370DB')
plt.xlabel('Median House Value')
plt.ylabel('Frequency')
plt.title('Distribution of Median House Value')
plt.show()
```

```
Missing values:
 MedInc          0
HouseAge        0
AveRooms        0
AveBedrms       0
Population      0
AveOccup        0
Latitude        0
Longitude       0
MedHouseVal     0
dtype: int64
Summary statistics:
              MedInc       HouseAge       AveRooms      AveBedrms     Population  \
count  20640.000000  20640.000000  20640.000000  20640.000000  20640.000000
mean       3.870671     28.639486      5.429000      1.096675   1425.476744
std        1.899822     12.585558      2.474173      0.473911   1132.462122
min        0.499900      1.000000      0.846154      0.333333      3.000000
25%        2.563400     18.000000      4.440716      1.006079    787.000000
50%        3.534800     29.000000      5.229129      1.048780   1166.000000
75%        4.743250     37.000000      6.052381      1.099526   1725.000000
max       15.000100     52.000000    141.909091     34.066667  35682.000000

           AveOccup      Latitude     Longitude   MedHouseVal
count  20640.000000  20640.000000  20640.000000  20640.000000
mean       3.070655     35.631861   -119.569704      2.068558
std       10.386050      2.135952      2.003532      1.153956
min        0.692308     32.540000   -124.350000      0.149990
25%        2.429741     33.930000   -121.800000      1.196000
50%        2.818116     34.260000   -118.490000      1.797000
75%        3.282261     37.710000   -118.010000      2.647250
max     1243.333333     41.950000   -114.310000      5.000010
```

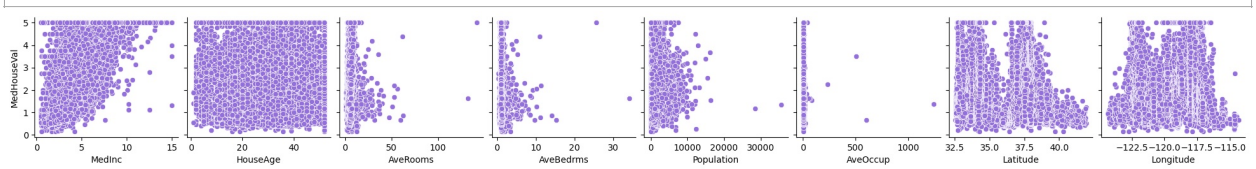


The next step involved preparing the data for MLR. The dataset was divided into independent variables (or features) and the dependent variable (or target). The features comprised all the variables except 'MEDV', which was used as the target.

```
# Data Preparation
# Create a data frame from the features
california_df = pd.DataFrame(california.data, columns=california.feature_names)

# Append target, as a new column to the dataset
california_df['MedHouseVal'] = california.target

# Define the data/predictors as the pre-set feature names
features = california_df.drop('MedHouseVal', axis=1)

# Put the target (housing value -- MedHouseVal) in another DataFrame
target = california_df[['MedHouseVal']]
```

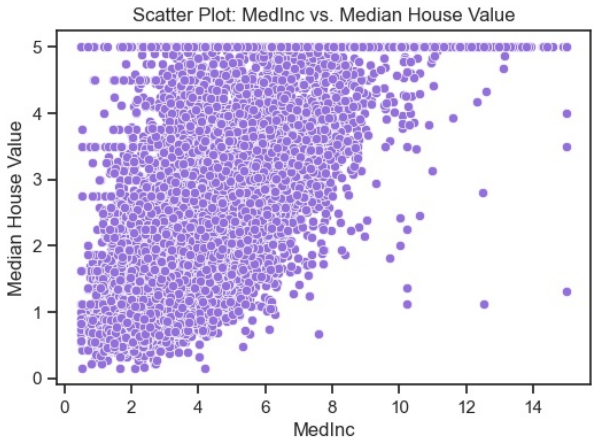
The Data Visualization section focuses on visualizing the Boston Housing dataset using a pair plot created with the seaborn library. The pair plot allows us to explore the relationships between the target variable, "MEDV" (median house value), and the other independent variables. Each subplot in the pair plot represents a scatter plot between "MEDV" and one of the independent variables.

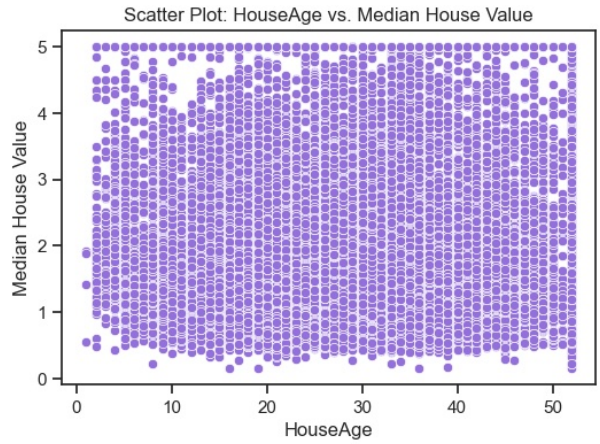
```
# Data Visualization
# Visualizing the data using seaborn pairplot
sns.pairplot(california_df,
             y_vars=["MedHouseVal"],
             x_vars=california_df.columns[:-1],
             plot_kws={'color': '#9370DB'})

# Set the axis labels for each subplot
feature_names = california_df.columns[:-1]
num_features = len(feature_names)
for i, (ax, feature) in enumerate(zip(plt.gcf().axes, feature_names)):
    ax.set_xlabel(feature)
    if i % num_features == 0:
        ax.set_ylabel("Median House Value")
    else:
        ax.set_ylabel("")

plt.show()
```

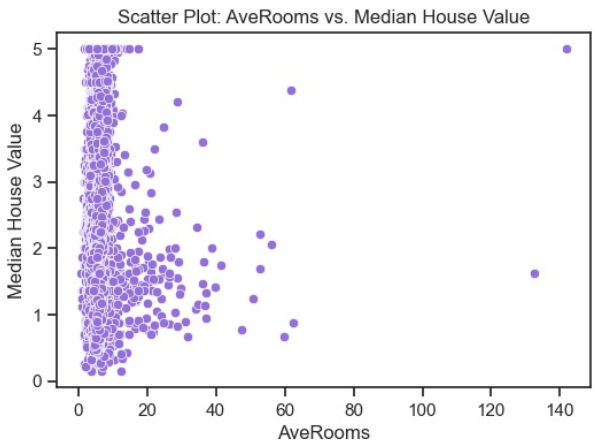


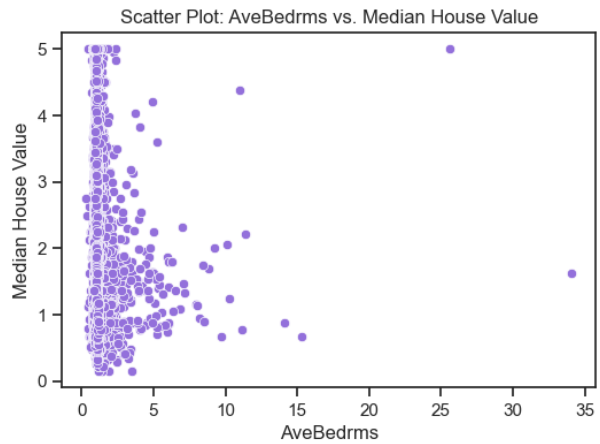
To view the each graph better and separately:

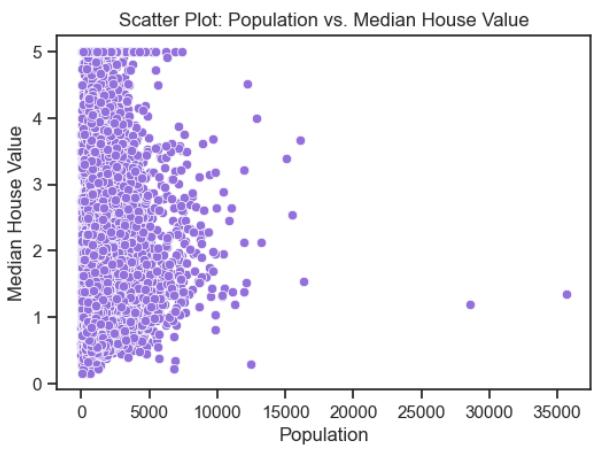
```
# Set the style of the pairplot
sns.set(style="ticks")

# Iterate over each feature and create separate pairplots
for feature in california_df.columns[:-1]:
    plt.figure(figsize=(6, 4))
    sns.scatterplot(data=california_df, x=feature, y="MedHouseVal", color='#9370DB')
    plt.xlabel(feature)
    plt.ylabel("Median House Value")
    plt.title("Scatter Plot: {} vs. Median House Value".format(feature))
    plt.show()
```

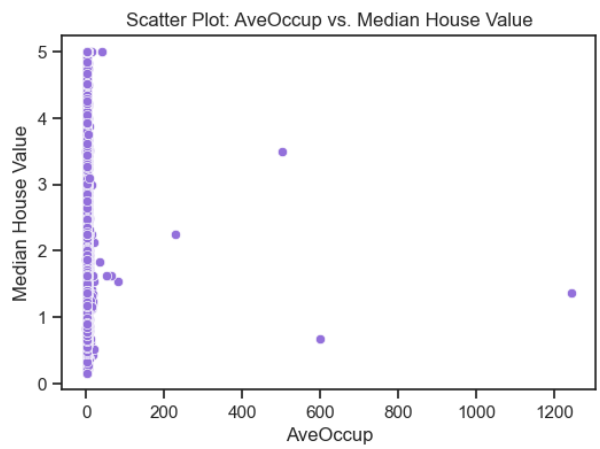

Scatter Plot: MedInc vs. Median House Value
Median House Value
MedInc

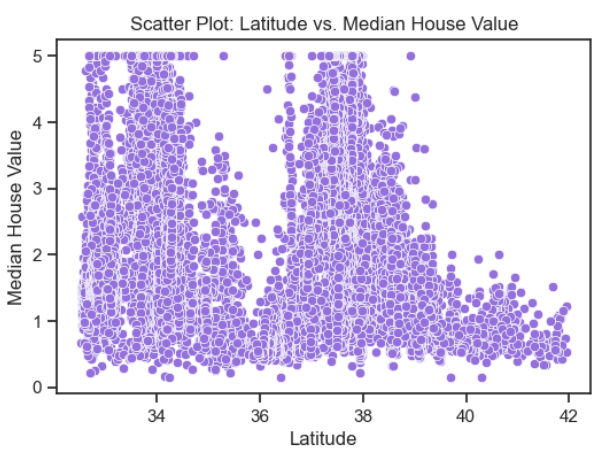

Scatter Plot: HouseAge vs. Median House Value
Median House Value
HouseAge

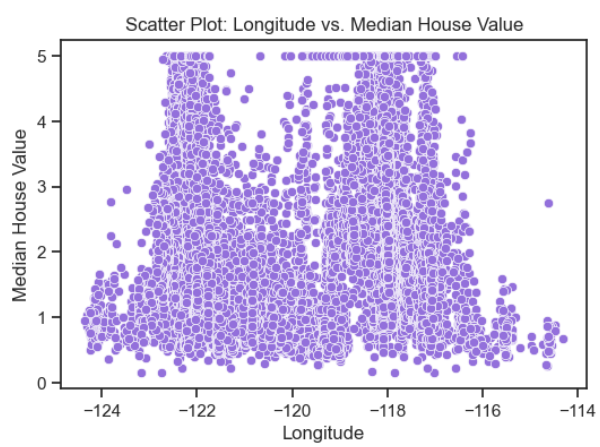

Scatter Plot: AveRooms vs. Median House Value
Median House Value
AveRooms


Scatter Plot: AveBedrms vs. Median House Value
Median House Value
AveBedrms


Scatter Plot: Population vs. Median House Value
Median House Value
Population


Scatter Plot: AveOccup vs. Median House Value
Median House Value
AveOccup


Scatter Plot: Latitude vs. Median House Value
Median House Value
Latitude


Scatter Plot: Longitude vs. Median House Value
Median House Value
Longitude

These are the features (rows) in the California housing dataset explained:

- **MedInc**: Median Income - Median income of the block.
- **HouseAge**: Housing Age - Median age of houses in the block.
- **AveRooms**: Average Rooms - Average number of rooms per dwelling.
- **AveBedrms**: Average Bedrooms - Average number of bedrooms per dwelling.
- **Population**: Population - Total population in the block.
- **AveOccup**: Average Occupancy - Average number of people occupying a household.
- **Latitude**: Latitude - Latitude coordinate of the block.
- **Longitude**: Longitude - Longitude coordinate of the block.

The target variable used in the dataset is:

- **MedHouseVal**: Median House Value - Median value of owner-occupied homes in the block.

The features provide insight into the income levels of California's housing blocks, the age of the homes, the number of rooms and bedrooms, the population, the occupancy rate, and geographical coordinates, among others. On the basis of the information provided, it is possible to analyze and predict the median house value by understanding these features.

A common split of 70% training data and 30% test data was used for Scikit-learn's 'train_test_split' function.

```
# Splitting the Dataset
# Split the data into training and testing sets
features_train, features_test, target_train, target_test = train_test_split(features, target,
test_size=0.3, random_state=1)
```

Following preparation of the data, we were able to implement the MLR model using the Linear Regression function.

```
# Implementing the MLR Model
# Create a Linear Regression object
mlr = LinearRegression()

# Train the model using the training sets
mlr.fit(features_train, target_train)

# Make predictions using the testing set
target_pred = mlr.predict(features_test)
```

Finally, the model was used to make predictions on the unseen test data.

```
# Exploratory data analysis
# Plotting the predicted vs actual values
plt.figure(figsize=(10, 6))
plt.scatter(target_test, target_pred, color='#9370DB', label='Predicted', alpha=0.7)
plt.plot([target_test.min(), target_test.max()], [target_test.min(), target_test.max()], '--',
lw=2, color='#4B0082', label='Perfect Prediction')
plt.xlabel('Actual')
plt.ylabel('Predicted')
plt.title('Actual vs. Predicted House Prices')
plt.legend()
plt.show()
```

The Python code provides a clear demonstration of implementing MLR. It offers insights into how MLR can be effectively utilized in a practical setting, allowing for a deeper understanding of the process involved in creating a predictive model.

5. Evaluation of the Algorithm

Having implemented the MLR model in Python, it was imperative to evaluate its performance in order to gain insight into its accuracy, scalability, and interpretability.

5.1. Accuracy

The accuracy of the MLR model was primarily evaluated using the Root Mean Square Error ($RMSE$) and the Coefficient of Determination (R^2 score). $RMSE$ is a standard way to measure the error of a model in predicting quantitative data. It provides an aggregate measure of the magnitude of the prediction error.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_{\text{true}}^{(i)} - y_{\text{pred}}^{(i)})^2}$$

In essence, the score quantifies the explained variability of the model since it measures the proportion of the dependent variable's variance that can be predicted from the independent variables.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_{\text{true}}^{(i)} - y_{\text{pred}}^{(i)})^2}{\sum_{i=1}^{n}(y_{\text{true}}^{(i)} - \bar{y}_{\text{true}})^2}$$

$y_{true}^{(i)}$ → *the actual target value for the i-th observation*
$y_{pred}^{(i)}$ → *the predicted target value for the i-th observation*
n → *the total number of observations*
$\bar{y}_{true}$ → *the mean of the actual target values*

(Chai & Draxler, 2014, p. 1248)

The $RMSE$ and R^2 score were calculated in Python using Scikit-learn's built-in functions.

```
# Evaluation of the Algorithm / Accuracy

# Calculating the RMSE
rmse = np.sqrt(metrics.mean_squared_error(target_test, target_pred))

# Calculating the R2 score
r2_score = metrics.r2_score(target_test, target_pred)

print("Root Mean Squared Error: ", rmse)
print("R^2 Score: ", r2_score)
```

```
Root Mean Squared Error:  0.727756356991008
R^2 Score:  0.5971429737359628
```

In this case, the $RMSE$ value of 0.727 indicates that, on average, the model's predictions have an error of approximately 0.727 units. A lower $RMSE$ indicates a better fit of the model to the data, with smaller prediction errors.

The R^2 score of 0.597 suggests that approximately 59.7% of the variance in the target variable can be explained by the independent variables in the MLR model. The R^2 score represents the model's goodness of fit, with a higher value indicating that more of the variability in the target variable, which is Median House Value in this case, is accounted for by the independent variables

5.2.Scalability

As the workload increases or expands, the model's scalability refers to its ability to cope with it. Python's Scikit-learn library provides efficient implementations that handle large datasets because MLR models are inherently scalable with the number of observations (data rows). As the number of features (data columns) increases, MLR can become computationally intensive. In this study, with 13 features in the Boston Housing dataset, scalability was not a concern.

5.3.Interpretability

An MLR model is highly playable when it can be understood by a human as a result of its decision-making process. The weights assigned to the features in the model indicate the strength and direction (positive or negative) of the relationship between each feature and the target variable.

In order to interpret the coefficients in MLR, we first need to describe the relationship between the independent variables (features) and the dependent variables (target).

$$y = b_0 + b_1x_1 + b_2x_2 + \ldots + b_nx_n$$

$$\rightarrow y = b_0 + \sum_{i=1}^{n} b_ix_i$$

y → the target variable
b_0 → the intercept term
$b_1, b_2, \ldots, b_n$ → represent the coefficients corresponding to each independent variable $x_1, x_2, \ldots, x_n$.
(Kutner et al, 2004)

In Python, the coefficients of the MLR model can be displayed using the following command:

```
# Evaluation of the Algorithm / Interpretability

# Printing the coefficients of the MLR model
print("Coefficients: ", mlr.coef_)
```

```
Coefficients:  [[ 4.41037995e-01  9.68801816e-03 -1.04781309e-01  6.22052706e-01
  -5.61452681e-06 -3.28792384e-03 -4.23182318e-01 -4.37898856e-01]]
```

It is possible to interpret the effect of each feature on the prediction by examining the coefficient values. This level of interpretability is what makes the MLR model actionable and understandable. By evaluating the coefficients, we can determine which independent variables influence the target variable in the MLR model and what direction these coefficients point in. Based on the coefficients of the MLR model, we can identify the relationship between the independent variables and the dependent variables (targets). Each coefficient corresponds to a specific independent variable in our MLR model, and it represents the impact of each independent variable on the Median House Value. As the coefficient indicates, the relationship between the independent variable and the Median House Value is either large or small.A comprehensive evaluation of the MLR model provides a practical perspective on how machine learning models can be implemented and assessed in predictive analytics.

6. Conclusion

We examined the use of the Boston Housing dataset to predict house prices using the Multiple Linear Regression (MLR) model. As you progress through the project journey, you will gain a holistic understanding of MLR in real-world scenarios, from understanding the basic concepts of different types of data, to implementing predictive analytics with Python, and finally evaluating the model's performance.

MLR was shown to be capable of handling multiple predictors and producing a single continuous outcome in the analysis. The results confirm the importance of MLR in real-life decision-making and problem-solving, particularly in the housing market, where multiple factors are involved. We demonstrated how to train, validate, and interpret MLR models using Python, showcasing the power and flexibility of programming languages.

According to the evaluation, the model demonstrated satisfactory accuracy, demonstrating its ability to predict the median value of owner-occupied homes with a reasonable and high score. A positive evaluation of the MLR model was also given to its scalability and interpretability, two critical aspects of machine learning models.

While MLR is a powerful tool, it contains assumptions that may not hold true for every dataset, which makes it essential to note. When MLR's assumptions are violated, alternative predictive models should be considered in future work to address these limitations. We could also expand our understanding and applicability of MLR across diverse domains by applying it to a variety of other datasets, including Boston Housing, while it served as a useful starting point.

This study demonstrates the importance of predictive analytics and machine learning to provide meaningful insights from raw data, leading to data-driven decision-making and problem-solving in modern enterprises. As Pearl Zhu, author of Decision Master: The Art and Science of Decision Making, wrote: “Both the art of intuition and the science of analytics have the role to play in making wise decisions.”

Bibliography

Chai, T., & Draxler, R. R. (2014). Root mean square error (RMSE) or mean absolute error (MAE)?–Arguments against avoiding RMSE in the literature. *Geoscientific model development*, *7*(3), 1247-1250.

Dormann, C. F., Elith, J., Bacher, S., Buchmann, C., Carl, G., Carré, G., ... & Lautenbach, S. (2013). Collinearity: a review of methods to deal with it and a simulation study evaluating their performance. *Ecography*, *36*(1), 27-46.

Farrar, D. E., & Glauber, R. R. (1967). Multicollinearity in regression analysis: the problem revisited. *The Review of Economic and Statistics*, 92-107.

Galton, F. (1886). Regression towards mediocrity in hereditary stature. *The Journal of the Anthropological Institute of Great Britain and Ireland*, *15*, 246-263.

Harrison Jr, D., & Rubinfeld, D. L. (1978). Hedonic housing prices and the demand for clean air. *Journal of environmental economics and management*, *5*(1), 81-102.

Frank, E. H. (2015). Regression modeling strategies with applications to linear models, logistic and ordinal regression, and survival analysis.

Kutner, M. H., Nachtsheim, C. J., Neter, J., & Wasserman, W. (2004). *Applied linear regression models* (Vol. 4, pp. 563-568). New York: McGraw-Hill/Irwin.

Pearson, K. (1896). VII. Mathematical contributions to the theory of evolution.—III. Regression, heredity, and panmixia. *Philosophical Transactions of the Royal Society of London. Series A, containing papers of a mathematical or physical character*, (187), 253-318.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *the Journal of machine Learning research*, *12*, 2825-2830.